

Lab 04 — Build Your Own Project

Let Claude interview you, design a mockup with you, and then build a personal app that fits what you actually want.

Introduction

Congratulations on finishing Labs 01 and 02, the core of this workshop! From here, it's your choice: continue to [Lab 03 — Build an Image-to-Calendar Assistant](#), build your own project with this guide, or both.

Your Claude session is now "context rich": it knows the apps you've built, how you like them set up and explained, and what worked along the way. That makes it a great starting point for something of your own.

In the earlier labs, we gave you a detailed prompt describing the app. This time, **you** are the product owner. Instead of a prompt with our idea of the app, you'll send a prompt that asks Claude to **interview you** first, write a short plan, and show you a **mockup** before it writes any code. That's how real product teams work, and it's a habit you can reuse for any idea after the workshop.

Before You Start

Your app will be different from everyone else's, and that's the point!

How this lab works:

- **Interview:** Claude asks you a handful of questions, with options to pick from.
- **Plan:** Claude writes a short MVP plan: what goes into the first version, and what can wait.
- **Mockup:** Claude shows you a picture of the main screen, with sample data. You ask for changes until it looks right.
- **Build:** only then does Claude build the app, set up the same way as your other lab apps.

Keep it small: aim for an app you could use next week, with one main screen and one main job. You can always add features later.

What You'll Need

- **Setup from Lab 00:** the Claude desktop app with a Cowork session connected to your lab folder, and Ollama installed and running. If you haven't done these yet, start with the [Lab 00 Setup guide](#).

- **The same Cowork session you used for Labs 01 and 02:** that's where all the context is. Your `My Building Skill.md` file from the earlier labs helps too.
- **AI model:** `qwen3.5:9b`, which you already downloaded in Lab 02. It can read both text and photos. If your laptop has only 8 GB of memory, use `qwen3.5:4b`.
- **Optional:** a phone camera, if your app reads photos (for example, a nutrition label or a vitamin bottle).

Step 01: Choose Your Project

Pick one of these ideas, or bring your own:

Project	What it could do	Ideas for photos or files
Fitness Tracker	Log workouts, steps or meals, and see your progress over time.	A photo of a nutrition label, or of a workout written on a gym whiteboard.
Medication Tracker	Keep a list of your medications, schedules and refill dates, plus your own notes.	A photo of a prescription bottle, or notes from a doctor's visit.
Portfolio Tracker	Track your investments, what they're worth, and how your mix has changed.	A CSV export from your brokerage account, or a statement.
Your Own Idea	Anything you'd like to have on your laptop.	Whatever your idea needs.

- **Use sample data in the lab:** Claude will create realistic, fictional sample data for testing. Real prescription labels and brokerage files contain personal information, so try those at home, if at all. A vitamin bottle makes a good stand-in for a prescription bottle.
- **Your data stays on your laptop:** like the earlier labs, these apps store your data locally and use an AI model that runs on your laptop.

Step 02: Send Your Interview Prompt

- **Before you send it:** check that your lab folder is connected to this Cowork session. If it isn't, add it again.

- **Copy the prompt for your project** below and send it. These prompts don't build anything yet: they start the interview.

Fitness Tracker

PROMPT

I'd like to build a personal fitness tracker for myself. Before writing any code, interview me so the app fits what I actually want.

- Ask me 5 or 6 questions, a few at a time. Where you can, give me options to choose from, and let me type my own answer. Cover what I want to track, how I'd like to get data in (typing, photos, files), what I want to see (for example, lists, charts, streaks or weekly summaries), and what should stay private.
- Ideas you can offer: workouts, steps, weight, meals, reading a nutrition label or a gym whiteboard from a photo, goals and progress charts.
- Then write a short MVP plan: the must-haves for a first version you can build in about 15 minutes, and ideas for later. Save it as `MVP Plan.md` in a new subfolder named `lab-04-fitness-tracker` inside my connected folder.
- Then create a mockup: a single `Mockup.html` page in that subfolder, showing the main screen with realistic sample data. It's a picture of the app, not the app itself, so don't write any Python yet. Show it to me and ask what I'd like to change.

Don't build the app until I tell you the mockup is approved.

Medication Tracker

PROMPT

I'd like to build a personal medication tracker for myself. Before writing any code, interview me so the app fits what I actually want.

- Ask me 5 or 6 questions, a few at a time. Where you can, give me options to choose from, and let me type my own answer. Cover who it's for (me, or someone I care for), what I want to track, how I'd like to get data in (typing, photos, files), what I want to see (for example, a daily schedule, refill dates or a printable list for my doctor), and what should stay private.
- Ideas you can offer: reading the label on a photo of a prescription bottle, organizing notes from a doctor's visit, a daily checklist, refill reminders, a list to bring to appointments.
- Then write a short MVP plan: the must-haves for a first version you can build in about 15 minutes, and ideas for later. Save it as `MVP Plan.md` in a new subfolder named `lab-04-medication-tracker` inside my connected folder.

- Then create a mockup: a single `Mockup.html` page in that subfolder, showing the main screen with realistic, fictional sample data. It's a picture of the app, not the app itself, so don't write any Python yet. Show it to me and ask what I'd like to change.

Include these limits in the plan and the mockup: this is a personal organizer, not medical advice. The app must never recommend doses or changes, or present drug interactions as fact, and it should remind me to check with my pharmacist or doctor. Anything read from a photo must be shown to me to review and correct before it's saved.

Don't build the app until I tell you the mockup is approved.

Portfolio Tracker

PROMPT

I'd like to build a personal investment portfolio tracker for myself. Before writing any code, interview me so the app fits what I actually want.

- Ask me 5 or 6 questions, a few at a time. Where you can, give me options to choose from, and let me type my own answer. Cover what I want to track (for example, stocks, funds or cash, across one or more accounts), how I'd like to get data in (typing, a CSV export, a statement), what I want to see (for example, total value, allocation charts or gains and losses), and what should stay private.
- Ideas you can offer: importing a CSV export from a brokerage, entering prices by hand, an allocation pie chart, a value-over-time chart, notes on why I bought something.
- Then write a short MVP plan: the must-haves for a first version you can build in about 15 minutes, and ideas for later. Save it as `MVP Plan.md` in a new subfolder named `lab-04-portfolio-tracker` inside my connected folder.
- Then create a mockup: a single `Mockup.html` page in that subfolder, showing the main screen with realistic, fictional sample data. It's a picture of the app, not the app itself, so don't write any Python yet. Show it to me and ask what I'd like to change.

Include these limits in the plan and the mockup: this is for tracking only, not financial advice. The app must never recommend buying or selling, and it must not connect to or trade in any brokerage account. For the first version, prices are entered by hand or imported from a file, unless I ask for a live price source.

Don't build the app until I tell you the mockup is approved.

Your Own Idea

Replace the text in brackets with a sentence or two about your idea.

PROMPT

I'd like to build [describe your idea, for example: an app that tracks my kids' chores and allowance]. Before writing any code, interview me so the app fits what I actually want.

- Ask me 5 or 6 questions, a few at a time. Where you can, give me options to choose from, and let me type my own answer. Cover who it's for, what it should do, how I'd like to get data in (typing, photos, files), what I want to see, and what should stay private.
- Then write a short MVP plan: the must-haves for a first version you can build in about 15 minutes, and ideas for later. Suggest a short app name, and save the plan as `MVP Plan.md` in a new subfolder inside my connected folder, named `lab-04-` followed by the app name (for example, `lab-04-chore-tracker`).
- Then create a mockup: a single `Mockup.html` page in that subfolder, showing the main screen with realistic, fictional sample data. It's a picture of the app, not the app itself, so don't write any Python yet. Show it to me and ask what I'd like to change.

If my idea involves health, money, or other people's personal information, add sensible limits to the plan (for example, no medical or financial advice) and tell me what they are.

Don't build the app until I tell you the mockup is approved.

Step 03: Answer the Interview

Claude now asks you its questions, a few at a time, usually with options you can click.

- **Pick an option, or type your own answer:** there are no wrong answers. You're describing the app you'd like.
- **Not sure?** Say "you decide" or "keep it simple". Claude will pick a sensible default and tell you what it chose.
- **Keep the first version small:** if Claude's plan has too many must-haves, ask it to move some to "ideas for later".
- **Read the MVP plan:** when the interview is done, Claude saves `MVP Plan.md` and summarizes it. Check that it describes what you want.

Step 04: Review the Mockup

Claude creates `Mockup.html` and shows it to you. It's a picture of your app's main screen, with sample data: nothing runs yet. To see it full size, open your project folder and double-click `Mockup.html`.

- **Ask for changes in plain language**, for example:

PROMPT

Move the chart to the top, and show this week's totals instead of the whole month.

PROMPT

Add a notes field to each entry, and make the refill dates stand out when they're less than a week away.

- **One or two rounds is usually enough.** The details are easier to change once the app works.
- **Happy with it?** Go to Step 05.

Step 05: Build Your App

- **Heads-up:** Claude takes about 10–15 minutes to build the app. Stay nearby, since it may pause for your approval. See **Step 06: While You Wait** below for what to do in the meantime.

Build Prompt

This prompt is the same for every project. It tells Claude to build the app you designed, set up the same way as your other lab apps.

PROMPT

The mockup is approved. Please build the app from `MVP Plan.md` and `Mockup.html`, in the same project subfolder. First read `My Building Skill.md` in my lab folder, if it's there, and follow it, and reuse what worked in the earlier labs in this session.

- Use Python 3.12+, uv for dependencies, and Streamlit for the interface. Configure Streamlit to always run on port 8504 (in `.streamlit/config.toml`), so it won't conflict with my other lab apps.

- Store my data locally, in a SQLite database in the project folder, so it's kept between app restarts. Include a clearly labeled button to load fictional sample data, so I can try the app right away.
- If the app uses AI (for example, to read a photo or to summarize my entries), use Ollama with the `qwen3.5:9b` model running on my laptop: no cloud AI services or external AI API calls. Show me anything the AI suggests, and let me review and correct it before it's saved.
- Keep all the limits in `MVP Plan.md`, and show them in the app where they matter.

Build the complete working project with clearly organized code and tests for the main features.

Make the app easy to start. Create a start file I can double-click, named after the app (for example, `Start Fitness Tracker.command` for macOS, made executable, and `Start Fitness Tracker.bat` for Windows). Each should install uv if it's missing (using uv's official standalone installer, not Homebrew) and call it by its full path, so I don't need to restart Terminal; install the dependencies; check that Ollama is running and download the model if it's missing; then start the app and open `http://localhost:8504` in my browser. Show clear progress messages, and explain how to stop the app.

Write two guides for a beginner, and keep both updated as the code changes. Save each one in the project folder as Markdown, HTML, and PDF (`Quickstart.md`, `Quickstart.html`, `Quickstart.pdf`, and `Application Guide.md`, `Application Guide.html`, `Application Guide.pdf`). In both, include instructions for both macOS (Terminal) and Windows (PowerShell), clearly labeled, wherever the steps or commands differ.

- **Quickstart:** title it "[App name] – Lab 04 Quickstart". Keep it to one page: a one-line summary, the app's folder and address (`http://localhost:8504`), how to start the app with the start file, how to stop and restart it, and the manual commands to use if the start file doesn't work.
- **Application Guide:** title it "[App name] – Lab 04 Application Guide". Explain what the app does and its limits, how it works in plain language, where my data is stored, which settings I can change, and how to run the tests and troubleshoot common problems.

When you're done, keep your final message to me short: tell me to open `Quickstart.html` and double-click the start file.

Step 06: While You Wait

Claude is now building your app. This takes about 10–15 minutes, and Claude shows its progress as it works. **Stay close to your laptop:** Claude sometimes pauses to ask a question or for your permission, and it waits until you answer.

- **Keep an eye on Claude:** glance at the Claude window every minute or two. If it asks for permission, read the request and click **Allow** (or **Allow for this task**, so it asks less often). If it asks a question, answer it. Nothing moves forward until you do.
- **Check your model:** there's nothing new to download. If you skipped Lab 02, open a terminal window (**Terminal** on a Mac, **PowerShell** on Windows) and run this command now:

```
ollama pull qwen3.5:9b
```

- **Plan your test:** look at the must-haves in `MVP Plan.md`. In Step 08, you'll check each one.
- **Compare notes with your neighbors:** see what they're building, and share ideas.
- **Feel free to take a quick break.** Check the Claude window as soon as you're back.

Step 07: Run Your App

When Claude has finished, your project folder (for example, `lab-04-fitness-tracker`) contains your app, a start file, and two guides: a **Quickstart** (how to start and stop the app) and an **Application Guide** (how the app works).

- **Open the Quickstart:** in your project folder, double-click `Quickstart.html`. It opens in your browser.
- **Start the app:** double-click the start file, `Start <your app name>.command` (Mac) or `.bat` (Windows). A terminal window opens and shows its progress. The first start takes a few minutes.
- **Open the app:** your browser opens `http://localhost:8504`. If it doesn't, type that address into your browser.
- **Keep the terminal window open:** the app runs as long as that window is open. To stop it, click the window and press **Control + C**.

Step 08: Try It

Your app is unique, so your `MVP Plan.md` is your checklist.

- **Load the sample data** and look around. Does the main screen look like your mockup?
- **Check each must-have** in `MVP Plan.md`: does it work the way you described?
- **Add an entry of your own**, then restart the app and check that it's still there.

- **If your app reads photos or files:** try one, and check that you can review and correct what the AI suggests before it's saved.
- **Check the limits:** for example, a medication tracker shouldn't give dosing advice, and a portfolio tracker shouldn't tell you what to buy.

When something isn't right, tell Claude what you expected and what happened, and fix one thing at a time.

Step 09: Save What You Learned as a Skill

Add this lab's lesson to the skill file you started in Lab 01. Send this prompt:

PROMPT

Update `My Building Skill.md` with what we learned in Lab 04: for a new idea, interview me first with a few multiple-choice questions, write a short MVP plan with must-haves and ideas for later, and show me a mockup with sample data before building. Also note any preferences I showed during the interview and mockup review. Keep everything that's already in the file. If the file doesn't exist yet, create it.

Next time you have an idea, ask Claude to read `My Building Skill.md` first, and it will start with the interview.

If You Get Stuck

If something fails, describe to Claude what you did, what you expected, and what happened. Include the error message when there is one. For example:

PROMPT

I clicked Save on a new entry, but it produced this error: [paste error]. Please help me fix it and update the Quickstart and Application Guide if any setup steps are missing.

Common problems:

- **Claude starts building before showing a mockup:** tell it, "Please stop. Show me the MVP plan and a mockup first, and wait for my approval."

- **The interview or the plan keeps growing:** tell Claude, "Let's keep the first version small. Move everything except the top three must-haves to ideas for later."
- **Claude says it can't find your folder:** add your lab folder to the session again, then tell Claude: "I've attached the folder now. Please put the project there."
- **Strange file errors, or the build keeps failing:** check where your lab folder is. If it's inside OneDrive, Dropbox, Box, Google Drive or iCloud (including a synced Desktop or Documents folder), the sync app can lock files while Claude writes them. Create a new folder in your home folder (see Lab 00), add it to the session, and ask Claude to build the project there.
- **The Mac won't open the start file:** right-click the `.command` file, choose **Open**, then click **Open** again. If it still won't run, ask Claude to make the start file executable.
- **"Can't reach Ollama":** open the Ollama app and check for the llama icon in the menu bar, then try again.
- **Reading a photo is slow:** the model takes a moment to load the first time. Later photos are faster.
- **Your laptop slows to a crawl or freezes:** close other apps first. If that doesn't help, ask Claude: "My laptop is struggling. Please switch the app to a smaller model, such as qwen3.5:4b, and update the guides." Or ask a lab proctor for a lab PC.
- **"Port 8504 is already in use":** the app is probably already running in another terminal window. Stop that one with **Control + C**, or use the one that's running.
- **Anything else:** copy the error message into Claude and ask for help, as in the example above.

Key Takeaways

In this lab, you were the product owner. Instead of handing Claude a finished design, you let it interview you, agreed on a small plan, and checked a mockup before any code was written. That's a habit worth keeping: a mockup takes a minute or two to change, while a finished app takes much longer.

You also saw how far a "context rich" session can take you. Claude reused what it learned in the earlier labs (how you like apps set up, started and documented) to build something new, quickly.

- **Next:** keep improving your app with the ideas below, or try [Lab 03 — Build an Image-to-Calendar Assistant](#) if you haven't yet.

Optional: Keep Going

Your `MVP Plan.md` has a list of ideas for later. Pick one, and add it the same way: describe it, ask for a quick mockup if it changes the screen, then build it. Make one change at a time and try the app again. For example:

PROMPT

Let's add the next idea from the "ideas for later" list in `MVP Plan.md`. Show me a quick mockup of the change first, then build it and update the guides.

PROMPT

Add a way to export my data to a CSV file, so I can keep a backup.

What's Next

- **Lab 03:** if you haven't done it yet, try [Lab 03 — Build an Image-to-Calendar Assistant](#).
- **Before you leave:** see [How to Save Your Work for Later](#), so you can pick up where you left off.